EXECUTIVE COMMITTEE | DECISION PAPER

Executive Brief

Modernising the Markets Confirmation Platform

Decision requested: approve phased migration from the Java 6 / WebLogic monolith to Spring Boot services with event-driven workflow.

14 yrs	38%	6–8 wks	3
CORE PLATFORM AGE	CHANGE SPEND ON LEGACY	TYPICAL RELEASE	MATERIAL EOL RISKS

Recommendation

Proceed with a strangler-pattern migration over four controlled waves. Establish a Kafka event backbone, extract bounded business capabilities into Spring Boot services, introduce service-owned data progressively, and retire WebLogic only after parallel-run evidence demonstrates functional and operational equivalence.

Why this matters now

- The current platform combines trade intake, confirmation decisioning, document generation, dispatch, inbound matching and exception handling in one deployment unit.
- Java 6 and WebLogic dependencies constrain security remediation, engineering productivity and infrastructure options.
- A single shared database makes change coordination expensive and increases blast radius during release and recovery.
- Business demand is shifting toward higher throughput, richer matching, faster rule change and better operational visibility.

Target architecture

BUSINESS CAPABILITY	TARGET SERVICE	PRIMARY INTERACTION
Trade intake & eligibility	Trade Intake / Decision	Kafka events
Validation & workflow	Confirmation Workflow	Kafka + API
Document creation & dispatch	Document / Dispatch	Kafka events
Inbound docs, matching, exceptions	Inbound / Match / Case	Kafka + operator APIs

Key design decisions

Strangler over big-bang rewrite

Allows business flows to be extracted one capability at a time while the current platform remains the system of record during transition.

Events for workflow; APIs for immediate queries

Kafka decouples long-running confirmation steps and enables replay. APIs remain appropriate for synchronous lookups or commands that require an immediate answer.

Local transactions + outbox + idempotency

Avoids distributed XA transactions and protects against duplicate or lost processing when database writes and event publication intersect.

Service-owned data as the target

Prevents a “distributed monolith” built around the existing shared database. Database separation will be sequenced behind capability extraction to control migration risk.

Exceptions as a first-class domain

Operational failures become visible cases with status, ownership, retry and audit—not merely application logs.

Four-wave migration

WAVE 1	WAVE 2	WAVE 3	WAVE 4
Foundation	**Low-coupling extraction**	**Core workflow**	**Inbound & retirement**
Spring Boot platform, CI/CD, Kafka, observability, security baseline	Document generation, dispatch, reference-data façade	Decision, validation, workflow state and exception case management	Matching, residual data migration, WebLogic shutdown

CI/CD and release controls

- Pipeline per service: compile, test, static analysis, SAST/dependency scan, contract tests, container build and image scan.
- Build once; promote the same immutable image through Development, SIT, UAT and Production.
- Database changes use Flyway/Liquibase with backward-compatible expand-and-contract migrations.
- Kafka schema compatibility and consumer contracts are checked automatically.
- Production deployment uses health gates, rollback criteria and telemetry.

Principal risks and mitigations

RISK	EXPOSURE	MITIGATION
Functional divergence	High	Golden-data regression, parallel run, outcome reconciliation
Duplicate / lost events	High	Idempotent consumers, outbox pattern, partitioning by business key
Shared-DB coupling	Medium	Ownership map, façade APIs, staged schema separation
Operational complexity	Medium	Tracing, SLOs, runbooks and dashboards

Decision requested

Approve

1) Four-wave target architecture and migration approach; 2) platform investment for Kafka, CI/CD and observability; 3) Wave 1 mobilisation; and 4) a quarterly executive checkpoint tied to decommissioning outcomes—not code-conversion activity.